DBMS Experiential Learning – Level 3 Project Synopsis

Team : Sonia Sharma - PES1UG24AM904

P Saanvi - PES1UG24AM905

1. Project Title

Academic Research Paper Repository with Semantic Search

2. Introduction

A web platform to store, search, and discover academic research papers using both keyword search and meaning-based (semantic) search, so relevant papers can be found even when exact keywords don't match.

3. Problem Statement

Keyword-based search fails when users phrase queries differently from a paper's actual wording, even if the topic is the same. Most repositories also treat papers as isolated records without capturing author, citation, and conference relationships properly.

4. Objectives

- Design a relational database for papers, authors, citations, conferences, users, and bookmarks.
- Support keyword/full-text search and semantic (vector-based) search over abstracts.
- Provide citation info and paper statistics.
- Justify combined use of SQL and vector databases.

5. Proposed Solution

PostgreSQL stores structured data (papers, authors, citations, conferences, bookmarks) with defined relationships and constraints. A vector database stores embeddings of paper abstracts. A user's natural-language query is converted to an embedding and matched via similarity search to retrieve conceptually related papers, alongside keyword/full-text search. A Next.js app with an API layer connects both databases.

6. Key Features

- Role-based login (Researcher, Admin)
- Add/manage papers, authors, conferences
- Keyword + full-text search
- Semantic search using vector similarity
- Citation tracking and counts
- Bookmarks and a statistics dashboard

7. Technology Stack

Frontend: Next.js | Backend: Next.js API routes | SQL Database: PostgreSQL | Vector Database: PostgreSQL (pgvector) / Pinecone / Qdrant | Version Control: Git & GitHub

8. Database Design / DBMS Concepts

Entities: Users, Papers, Authors, Paper_Authors, Conferences, Citations, Bookmarks — linked via primary/foreign keys with constraints. Abstract embeddings stored in the vector DB with paper ID as metadata. Advanced concepts used: transactions, views, triggers (auto-update citation counts), indexing, full-text search, and vector similarity search. SQL handles structured, relationship-heavy data; the vector DB handles unstructured text meaning — justifying the dual-database approach.

9. Expected Outcome

A working web application enabling users to search and discover research papers via keyword and natural-language queries, demonstrating justified SQL + vector database integration and advanced DBMS features, going beyond basic CRUD.